

AI 및 엔진 활용 기술 설명서

20.26 THE ARCHIVE — 2026 밈 × 20.26초 물리 액션 게임

플랫폼

웹 브라우저 (설치 불필요)

엔진

Phaser 3.90.0 + Matter.js

수록 미니게임

9종

작성일

2026-09-06

1 프로젝트 및 기술 개요

2026년의 화제와 밈을 **20.26초** 제한의 짧은 물리 액션 스테이지로 재해석한 웹 게임이다. 플레이어는 3개 ACT × 6판, 총 18개의 기록을 모아 엔딩에 도달한다. 물리는 학습 소재가 아니라 "이렇게 하면 이렇게 될 것 같다"는 예측과 그 예측을 배신하는 결과를 만들어내는 게임 메커니즘으로 사용한다.

렌더 · 게임 루프	Phaser 3.90.0 (로컬 번들, CDN 의존 없음) · 960×540 논리 해상도 · Scale.FIT
물리	Matter.js 강체 물리(Phaser 내장, 메차 쌓기·두꺼꾸 새총) + 게임별 전용 물리 코어(자체 구현)
UI · 연출	DOM/CSS 레이어(12종 CSS, 약 4,300줄) — 메인 메뉴 · 컷신 · HUD · 도감 · QA 모드
코드 규모	애플리케이션 JS 약 8,500줄(생성 번들 제외) · index.html 962줄
에셋	이미지 245장(전량 GPT 생성) · BGM(Suno) · 효과음(Pixabay 등)
빌드 · 검증	PowerShell 스크립트 14종(에셋 베이킹 · 번들 빌드) · 자동 검증 29종(헤드리스 Chrome)
AI 활용	이미지(GPT) · 음악(Suno) · 코드 구현 및 검증(GPT, Claude)

설계 원칙: "표시와 판정의 분리"

AI로 생성한 그림은 **보이는 것만** 바꾸고 **판정**은 바꾸지 않는다. 이미지 교체·재생성이 난이도에 영향을 주지 않으므로 에셋을 여러 번 다시 뽑아도 밸런스를 다시 잡을 필요가 없다. 이 원칙 덕분에 짧은 개발 기간 안에서 "물리 규칙 확정 → AI 에셋 반복 생성"을 병렬로 진행할 수 있었다. (유일한 예외는 메차 쌓기로, 그림의 알파 채널에서 충돌체를 자동 추출한다 — 3.3 참고.)

2 엔진 활용 기술

2.1 단일 씬 + 스테이지 레지스트리 구조

9개 미니게임이 Phaser 씬 하나(ArchiveGame)를 공유한다. 각 게임은 build / update / render와 선택적 action · press · pointerDown/Move/Up · cancelInput · timeout · dispose 혹은 구현하는 데이터 객체이며, STAGE_GAMES 레지스트리에 ID로 등록된다. 게임 파일끼리는 서로를 참조하지 않는다.

- 공통 씬이 입력·타이머·물리 스텝·결과 이벤트를 전담하므로, 새 게임 추가 비용이 파일 한 개 + 등록 한 줄이다.
- 스테이지 전환 시 표시 객체·트윈·타이머·마스크·입력 상태를 일괄 정리하고, Matter 월드는 dispose에서 파기한다.
- 렌더·충돌 보조 도구는 minigame-kit.js(MINI)에만 두고, 게임 상태는 매판 새로 만든 scene.state에 격리한다.

2.2 120Hz 고정 타임스텝 — 20.26초 규칙의 기술적 근거

"20.26초"는 이 게임의 정체성이므로 화면의 숫자와 실제 물리 진행이 어긋나면 안 된다. 공통 루프는 프레임 델타를 누적해 **1/120초 고정 스텝**으로 분할 소비하며, 같은 스텝 안에서 물리 갱신 · 남은 시간 차감 · HUD 송출이 함께 일어난다.

- 모니터 주사율(60/120/144Hz)이 달라도 낙하 거리·점프 높이·클리어 시간이 동일하다.
- 중력은 $y += v \cdot dt + g \cdot dt^2 / 2$ 로 정적분해 적분 오차로 인한 "닿을 듯 말 듯"한 점프 실패를 없앴다.
- 남은 시간이 스텝보다 작으면 그만큼만 소비해 20.26초 경계에서 물리가 한 프레임 더 진행되지 않는다.
- 왕사남 호랑이 추격의 벽 충돌 시간 차감(-1초) 같은 페널티도 같은 시간축에 반영되어 HUD·책상 시계와 항상 일치한다.

2.3 두 갈래 물리 구현 — 강체 시뮬레이션과 전용 물리 코어

방식	적용	내용
Matter.js 강체	메차 쌓기 두쫘쿠 새총	"결과를 예측할 수 없어야 재미있는" 게임에 사용. 메차 쌓기는 사람 모형이 실제 접촉점·질량중심·관성·마찰에 따라 회전하고 무너지며, 두쫘쿠 새총는 목조 구조물의 기둥·바닥·지붕이 충격으로 파손되어 잔해로 남는다. Matter의 속도 단위(60Hz 프레임 이동량)를 초당 픽셀로 환산해 120Hz 공통 스텝과 정합시켰다.
전용 물리 코어	거제, 야호 / 왁뿌볼 살리기 / 왕사남 호랑이 추격 / 회전 고양이 / 월드컵 조추첨 / 거미줄 질주 / 피겨 암호	"규칙이 곧 재미인" 게임에 사용. 원-사각 최근접점 충돌과 침투 해소를 최대 3패스 반복하고, 벽 접촉을 래치로 유지해 모서리에서 충돌이 중복 계산되지 않게 한다. 튜닝값이 게임마다 명시적으로 노출되어 밸런스 조정이 수치 한 줄 수정으로 끝난다.

두 방식 모두 각 게임의 tuning 블록으로 조정한다. 예: 왁뿌볼 살리기 `jump 740 · jumpDecay 0.9 · minJump 300`(점프마다 점프력 90%로 감소), 거미줄 질주 `boost 35% · maxMultiplier 3배 · gravity 1050/1600~2400`, 회전 고양이 `minGap 152px · spawnAhead 880px`.

2.4 "이상 현상" — 조작할수록 물리가 변하는 공통 규칙

모든 미니게임은 입력 횟수가 물리 상수를 바꾸는 공통 장치를 갖는다. 반전할수록 장애물이 더 많이 떨어지고(거제, 야호), 점프할수록 점프력이 줄고(왁뿌볼 살리기), 방향을 바꿀수록 가속·마찰이 줄고(왕사남 호랑이 추격·피겨 암호), 실패할수록 톨렛 마찰이 감소하며(월드컵 조추첨), 새 연결점마다 35%씩 가속되는 대신 낙하가 강해진다(거미줄 질주). 20.26초라는 짧은 시간 안에 "익숙해질 무렵 규칙이 어긋나는" 체험을 만들기 위한 설계이며, HUD의 위험도 막대가 그 누적치를 실시간으로 보여 준다.

3 AI 활용 기술 — 에셋 생성 파이프라인

게임에 들어간 이미지 245장은 **전량 GPT로 생성**했다. 다만 생성 이미지를 그대로 게임에 넣으면 해상도·여백·회전축이 제각각이라 판정이 흔들린다. 그래서 "AI 생성물 → 게임 텍스처"를 사람이 손으로 다듬지 않고 **스크립트가 규격화**하는 파이프라인을 구축했다.

STEP 1 GPT 생성 역할별 프롬프트로 투명 배경 PNG 생성. 캐릭터는 자세 세트 단위(달리기 6 프레임 등)로 일괄 생성.	STEP 2 베이킹 PowerShell + System.Drawing 스크립트 14종이 여백 절단·중심 정렬·엷지 블리딩·리사이즈 수행.	STEP 3 WebP 인코딩 알파를 보존한 채 WebP로 압축. 원본 PNG 151장 → 게임용 WebP 116장.	STEP 4 매니페스트 연결 역할 이름(ball11, backdrop ...)으로만 참조. 엔진은 게임명과 역할로 읽는다.	STEP 5 헤드리스 검증 실제 Chrome에서 텍스처 적재·표시 크기·회전축을 검사하고 스크린샷을 남긴다.
--	--	--	--	--

3.1 생성 이미지를 게임 규격으로 굽는 베이킹 스크립트

AI 생성 이미지의 편차를 흡수하기 위해 스크립트가 자동으로 처리하는 항목은 다음과 같다.

처리	목적과 방식
알파 기준 여백 절단	그림마다 다른 투명 여백을 잘라내, 화면에 보이는 실루엣과 판정 크기를 일치시킨다.
공통 사각형 정렬	달리기 6프레임을 하나의 공통 프레임 으로 잘라 굽는다. 걸음이 넘어가도 키와 발끝이 흔들리지 않는다.
회전축 보정	공(악뽀뽀 살리기)은 가장 넓은 가로줄에서 지름을 재어 그 중심을 캔버스 중앙에 배치. 굴러도 회전축이 튀지 않는다.
엷지 블리딩	가장자리 색을 투명 쪽으로 번지게 해 WebP 손실 압축이 실루엣에 테두리를 남기지 않게 한다.
밝기 변형 사전 생성	런타임 tint가 무시되는 환경(file:///의 Canvas 렌더러)을 위해 -dim 변형을 미리 구워 둔다.

3.2 도구 부재를 우회한 인코딩 — 이미 설치된 Chrome을 인코더로 사용

개발 환경에 Python·ImageMagick·libwebp가 없어 일반적인 이미지 파이프라인을 구성할 수 없었다. 이를 PowerShell + .NET System.Drawing (픽셀 처리)과 헤드리스 Chrome의 canvas.toDataURL('image/webp', q) (인코딩)로 대체했다. 추가 설치 없이 팀원 누구나 같은 결과물을 재현할 수 있고, 알파 채널도 그대로 보존된다.
--

3.3 AI 그림에서 충돌체를 자동 추출 (메차 쌓기)

"사람을 쌓는" 게임인 메차 쌓기만은 그림 모양이 곧 게임성이라 표시-판정 분리 원칙의 예외를 두었다. bake-stack-poses.ps1이 GPT가 생성한 8가지 자세 PNG의 **알파 채널을 격자로 훑어** 불투명 영역만 덮는 사각형 충돌 조각을 자동 생성하고, 이를 pose-shapes.js로 출력한다.

- 격자 셀 4.2px, 자세당 최대 20조각으로 실루엣 **89~100%**를 덮는다.
- 팔다리 사이의 빈 공간은 실제로 비어 있어, 다른 사람 모형의 팔이 그 틈으로 들어간다 — 그림대로 쌓인다.
- 그림을 다시 생성하면 스크립트만 재실행하면 되고, 충돌체를 손으로 다시 그릴 필요가 없다.
- 8가지 자세 × 7가지 낙하 각도가 어긋나며 순환해 같은 조합은 56판에 한 번만 등장한다.

3.4 밈 텍스트를 장애물로 쓰는 런타임 생성 (회전 고양이)

회전 고양이의 장애물은 이미지가 아니라 웹 폰트로 조판된 밈 문장이다. "여러분 → 저 됐어요 → X됐어요 → 샤갈! → 야르~"를 한 글자씩 세로로 세워 천장·바닥에 번갈아 붙인다. 판정 폭은 실제로 그려진 글자 폭에서 뺄으므로 **밈 문구만 바꾸면 새 장애물이 되고, 맞춤 수치가 없다**. 폰트가 늦게 도착하면 그 시점에 다시 재어 그림과 판정을 맞춘다. 통로가 minGap(152px)보다 좁아지지 않도록 기둥 높이를 자동으로 제한해 클리어 가능성을 보장한다.

4 AI 활용 기술 — 개발 및 검증

4.1 AI 코딩 도구의 역할 분담

코드 구현에는 **OpenAI Codex**와 **Claude Code**를 함께 사용했다. 3인 팀이 158개 커밋 동안 9개 미니게임과 스토리·UI·QA 시스템을 완성할 수 있었던 것은, AI 도구가 다음 작업을 맡았기 때문이다.

영역	AI가 수행한 작업
게임 로직 구현	물리 규칙(진자 운동, 원-사각 충돌 해소, 중력 정적분)과 스테이지별 <code>build/update/render</code> 구현
밸런스 조정	"무충돌 클리어 약 16.3초" 같은 목표치를 두고 튜닝값을 반복 조정 — 자동 검증으로 즉시 확인
에셋 파이프라인	PowerShell 베이킹 스크립트 14종 작성. 알파 추출·회전축 보정 등 규칙을 코드로 고정
검증 자동화	Chrome DevTools Protocol 기반 검증 스크립트 29종 작성 (4.2)
문서화	모듈·게임별 README와 코드 주석. "왜 이 수치인가"를 코드 옆에 남겨 팀 인수인계 비용을 낮춤

AI가 생성한 코드는 전부 자동 검증(4.2)과 실제 플레이 확인을 통과한 것만 병합했다. 밸런스의 최종 판단은 사람의 플레이테스트로 했다.

4.2 헤드리스 Chrome 기반 실행 자동 검증

Node.js를 쓸 수 없는 환경이라, **PowerShell**이 **WebSocket**으로 **Chrome DevTools Protocol**에 직접 연결하는 검증 체계를 만들었다. 모킹이 아니라 **실제 Phaser 실행**을 실제 브라우저에서 돌리고 실제 입력을 넣는다.

검증 항목	클리어 가능성 검사
9개 게임 로딩 · ACT별 랜덤 6개 중복 방지	단순히 "오류가 없는가"가 아니라 "자동 입력으로 실제 클리어되는가" 를 검사한다. 왁뿌볼 살리기는 최저 점프력에서
20.26초 종료 · 성공 경로 · 페널티 유지	계단 끝 6px 지점 도약은 성공하고 24px 일찍 뛰면 실패하는 것까지 확인하며, 두뽀쿠 새총은 실제 마우스 조작으로
실제 키보드 입력(D + Space)으로 착지 검사	지지대를 부순 뒤 잔해 사이의 남은 목표를 맞힌다. AI가 밸런스를 조정할 때마다 이 검사가 "여전히 깰 수 있는 판인가"를 즉시 답해 준다.
축소된 모니터 화면에서의 마우스 드래그	
일시정지 · 실행 전환 시 자원 정리 누수	
롤렛 확률 · 최고 기록 · 스토리 분기	
메차 쌓기 전용: 충돌체와 그림의 크기 일치, 잔해 위 재쌓기	

검증 결과는 스크린샷으로 `tests/.artifacts/`에 남는다(점프 횟수별 파손 상태, 쌓인 탑, 넘어진 탑 등). 자동 입력 성공이 사람 기준 난이도를 보증하지는 않으므로, 최종 난이도는 플레이테스트로 확정했다.

4.3 재현 가능한 실패 — 시드 기반 결정론

스테이지 구성은 `seededRandom`(시드 난수)으로 생성한다. 실패 후 재시도하면 **같은 스토리 시드로 맵과 목표가 그대로 복원**되어 "방금 그 판"을 다시 도전하게 되고, ACT를 새로 시작할 때만 새로 추가한다. 버그 재현과 자동 검증의 안정성도 여기서 나온다.

5 실행 환경 및 배포 기술

과제	해결 방식
설치·서버 없이 실행	브라우저가 <code>file://</code> 에서 ES 모듈 <code>import</code> 를 막으므로, 빌드 스크립트가 18개 모듈의 <code>import/export</code> 를 제거해 하나의 클래식 스크립트 번들(<code>game-classic.js</code>)로 합친다. <code>index.html</code> 을 더블클릭하는 것만으로 전체 게임이 실행된다.
렌더러 폴백	<code>file://</code> 에서는 WebGL 텍스처 업로드가 차단되므로 자동으로 <code>Phaser.CANVAS</code> 와 <code>HTMLImageElement</code> 로더로 전환하고, 웹 서버 환경에서는 <code>Phaser.AUTO</code> (WebGL)를 쓴다.
해상도 대응	설계 좌표(가로 20~940)는 고정된 채 카메라 줌 한 번으로 16:9 화면을 꽉 채운다. 게임 좌표와 판정은 전혀 바뀌지 않고 위아래로 더 보이게 되므로, 어떤 모니터에서도 난이도가 동일하다.
엔진 ↔ DOM 연동	Phaser 씬과 DOM UI는 <code>window</code> 커스텀 이벤트(<code>archive-hud</code> , <code>archive-stage-end</code> , <code>archive-respawn</code> ...)로만 통신한다. 남은 시간·조작 횟수·이상 현상은 DOM HUD가 그리고, 캔버스 밖 "책상 위의 손" 연출도 같은 신호로 반응한다.
진행 저장 · QA	<code>localStorage</code> 에 진행도·최고 기록·도감을 저장하되 접근 실패 시 세션 저장으로 자동 강등. QA 모드는 제한시간·아트 세트를 고정해 검수할 수 있으며 QA 기록은 정식 기록에 남기지 않는다.

6 정리 — 기술 선택의 근거

선택	얻은 것
Phaser 3 + 단일 씬 구조	게임 1개 추가 비용 = 파일 1개 + 등록 1줄. 짧은 기간에 9개 미니게임 확보
120Hz 고정 타임스텝	기기와 무관하게 동일한 20.26초. 화면의 숫자와 물리가 어긋나지 않음
표시-판정 분리	AI 에셋을 몇 번을 다시 뽑아도 밸런스 재조정 불필요 → 아트와 로직의 병렬 진행
스크립트 기반 에셋 베이킹	AI 생성물의 편차(여백·해상도·회전축)를 사람 손 없이 규격화. 재생성 비용이 거의 0
실브라우저 자동 검증	AI가 조정한 밸런스가 "여전히 깰 수 있는 판인가"를 즉시 확인 → 빠른 반복
단일 HTML 실행	심사·시연 환경에서 설치나 서버 구성 없이 즉시 플레이 가능

AI 활용 범위 명시

이미지 에셋(캐릭터·배경·오브젝트 245장)은 전량 GPT로 생성했다. 코드는 OpenAI Codex와 Claude Code를 활용해 구현하고, 자동 검증과 플레이테스트를 통과한 결과만 반영했다. BGM은 Suno를 활용해 제작하고, 효과음은 Pixabay 등 무료 음원 라이브러리에서 확보했으며, 항목별 출처와 이용조건을 [sounds/](#)의 관리표(CSV)에 기록하고 있다. 실존 인물·사건은 직접 조롱하지 않고 행동과 mim의 구조만 패러디하며, 실제 피해가 발생한 사건은 소재에서 제외했다.

7 팀 구성 및 AI 활용 설계

7.1 팀원별 역할과 활용 AI 도구

3인 팀이 기획 분야별 역할을 분담하고, 각 담당 영역의 개발에도 함께 참여했다. AI 도구는 콘텐츠 구체화, 아트·사운드 제작, 코드 구현과 검증 과정에 활용했다.

팀원	담당 역할	활용 AI 도구
박수찬	콘텐츠 기획, 사운드, 개발	GPT, Suno
성민아	UI 기획, 아트, 개발	GPT, Claude
이상호	시스템 기획, QA, 개발	GPT, Claude

7.2 회차 진행에 따른 자동 난이도 상승 설계

회차가 진행될수록 AI가 난이도를 자동으로 높이도록 설계했다. 반복 플레이에서도 도전이 이어지도록, 회차 진행과 스테이지 난이도 조정을 연결하는 방향이다.

7.3 스테이지 맵과 구성요소의 자동 변동 설계

각 스테이지의 맵과 구성요소를 AI를 활용해 자동으로 변동할 수 있도록 설계했다. 같은 미니게임에서도 서로 다른 구성을 경험하도록 하여 반복 플레이의 다양성을 높이는 것을 목표로 한다.

위 두 항목은 AI 활용 설계 방향을 설명한다. 앞서 설명한 시드 기반 구성 생성 및 개발 과정의 AI 밸런스 조정과 구분하며, 실시간 AI 호출 방식이나 구현 완료 범위를 특정하지 않는다.

7.4 수록 미니게임 · 9종

게임명	게임명	게임명
거제, 야호	왁뿌볼 살리기	메차 쌓기
왕사남 호랑이 추격	두쫘꾸 새총	회전 고양이
월드컵 조추첨	거미줄 질주	피겨 암호

AI 활용 범위

GPT와 Claude를 기획·개발 및 검증에 활용하고, 이미지 에셋은 GPT로 생성했다. BGM 제작에는 Suno를 활용했으며, 효과음은 Pixabay 등 음원 라이브러리를 활용했다. 생성 결과의 게임 적합성과 최종 밸런스는 팀원이 검토하고 플레이테스트를 통해 판단한다.